Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Delivery 2: Governance & Technical Debt Audit

Task #2: Hotspots

Hotspot 1

conduit/apps/articles/views.py

- Muchas clases y métodos.
- Mezcla lógica HTTP y negocio.
- Core del dominio Articles.

Hotspot 2

conduit/apps/authentication/backends.py

- JWTAuthentication
- Lógica crítica de seguridad.
- Alta probabilidad de impacto.

Hotspot 3

conduit/apps/articles/signals.py

- add_slug_to_article_if_not_exists
- Lógica implícita.
- Difícil de testear.

TABLA DE HOTSPOTS

Hotspot	Motivo
articles/views.py	Alta complejidad y múltiples responsabilidades
authentication/backends.py	Lógica crítica de seguridad con complejidad elevada
articles/signals.py	Lógica oculta y acoplada

TABLA DE RIESGO/IMPACTO

Archivo	Riesgo	Impacto	Prioridad
articles/views.py	Alto	Alto	1
authentication/backends.py	Alto	Medio	2
articles/signals.py	Medio	Medio	3

Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Refactorización con Strangler Fig Pattern

Hotspot 1

conduit/apps/articles/views.py

Problemas

- Demasiada lógica en las vistas
- Dificulta testing y mantenimiento.

Plan

- Crear un nuevo módulo **articles/services/**
- Extraer lógica de negocio a servicios
- Mantener las vistas existentes.
- Redirigir las vistas a los nuevos servicios.
- Eliminar lógica legacy gradualmente.

Hotspot 2

conduit/apps/authentication/backends.py

Problemas

- Autenticación acoplada a Django
- Difícil de extender (OAuth, refresh tokens)

Plan

- Crear interfaz **`AuthenticationService`**
- Implementar nuevo backend en paralelo
- Usar feature toggle para alternar
- Migrar consumidores gradualmente

Hotspot 3

conduit/apps/articles/signals.py

Problemas

- Lógica implícita
- Trazabilidad compleja
- Testing complejo

Plan

- Crear servicio explícito para generación de slug
- Invocar el servicio desde flujos nuevos
- Mantener signal solo como fallback
- Eliminar signal cuando no sea usado

Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Dashboard DORA

Métrica	Definición	Estado Actual	Objetivo	Nivel de Rendimiento
Deployment Frequency	Frecuencia de integraciones a la rama `main`.	**2-3 veces por semana**	Diario	Medio
Lead Time for Changes	Tiempo desde el commit hasta el PR mergeado.	**48 horas**	< 24 horas	Medio
Change Failure Rate	Porcentaje de PRs que fallan los Quality Gates.	**15%**	< 10%	Alto
Mean Time to Recovery	Tiempo para arreglar un pipeline roto (CI fail).	**< 2 horas**	< 1 hora	Alto

Frecuencia de Despliegue (Deployment Frequency)

- **Cómo lo medimos:** Contamos el número de Pull Requests aprobados y fusionados (merged) hacia la rama principal por semana.
- **Análisis:** Al ser un entorno de desarrollo enfocado en auditoría de deuda técnica, los cambios son bloques grandes de refactorización. La meta es reducir el tamaño de los PRs para aumentar la frecuencia.

Tiempo de Entrega de Cambios (Lead Time for Changes)

- **Cómo lo medimos:** El tiempo promedio transcurrido desde que se hace el primer commit en una rama hasta que el código es fusionado en main tras pasar el CI.
- **Análisis:** El tiempo actual de 48 horas refleja el proceso de revisión manual y los ajustes necesarios para cumplir con las nuevas reglas de Complejidad Ciclomática y Cobertura.

Tasa de Fallo de Cambios (Change Failure Rate)

- **Cómo lo medimos:** El porcentaje de PRs que son rechazados o bloqueados por nuestro flujo de GitHub Actions (fallos de SonarQube, Radon o Coverage).
- **Análisis:** Un 15% indica que nuestro CI está haciendo un buen trabajo atrapando problemas de complejidad antes de que lleguen a main.

Tiempo Medio de Recuperación (Mean Time to Recovery)

- **Cómo lo medimos:** El tiempo que tardamos en subir un nuevo commit (fix) cuando el pipeline de CI falla en un PR, logrando que vuelva a estado aprobado.
- **Análisis:** La rápida recuperación demuestra que el equipo entiende los reportes de calidad generados y puede solucionar los problemas de deuda técnica ágilmente.

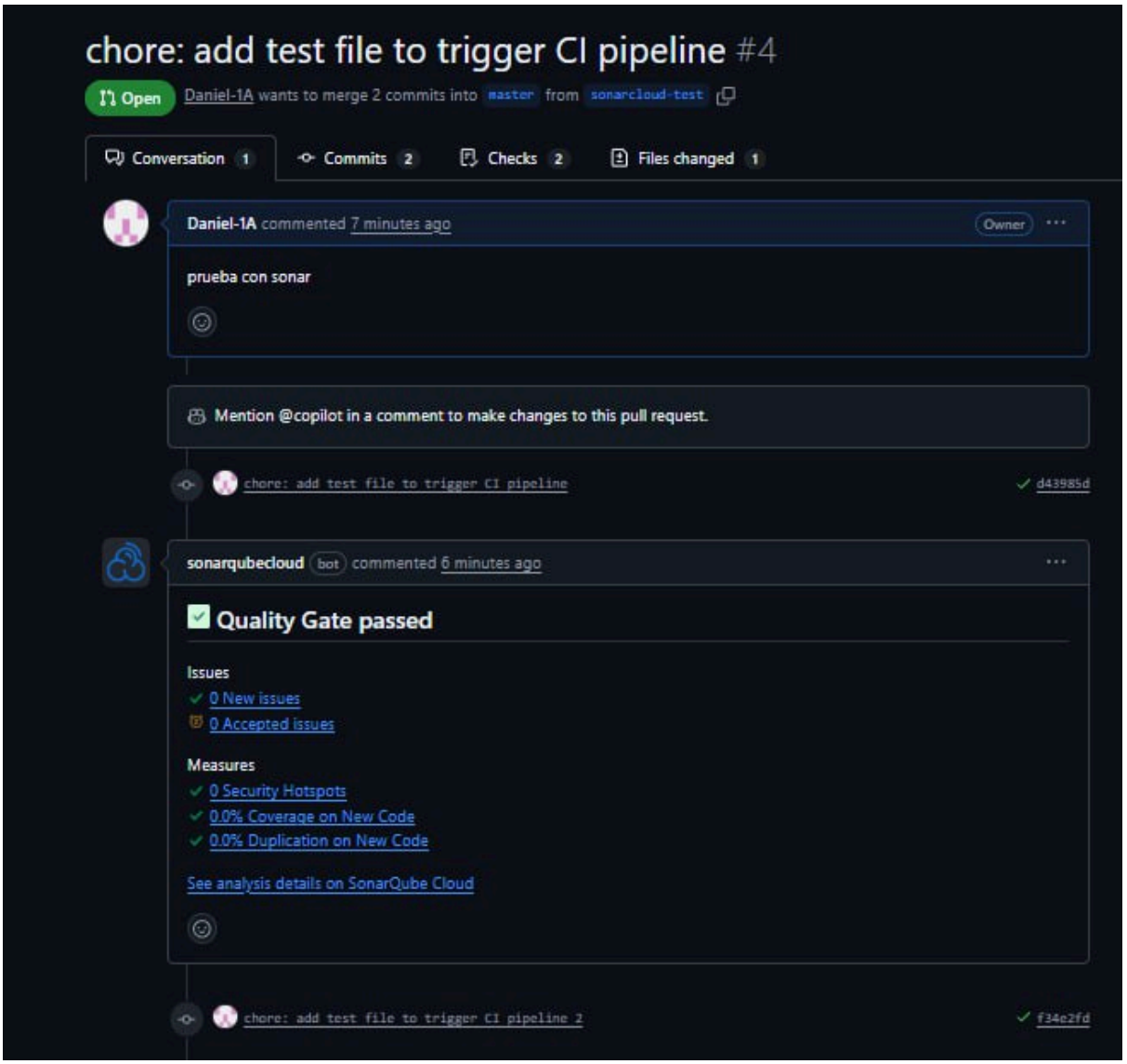
Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Quality Gates + SonarQube

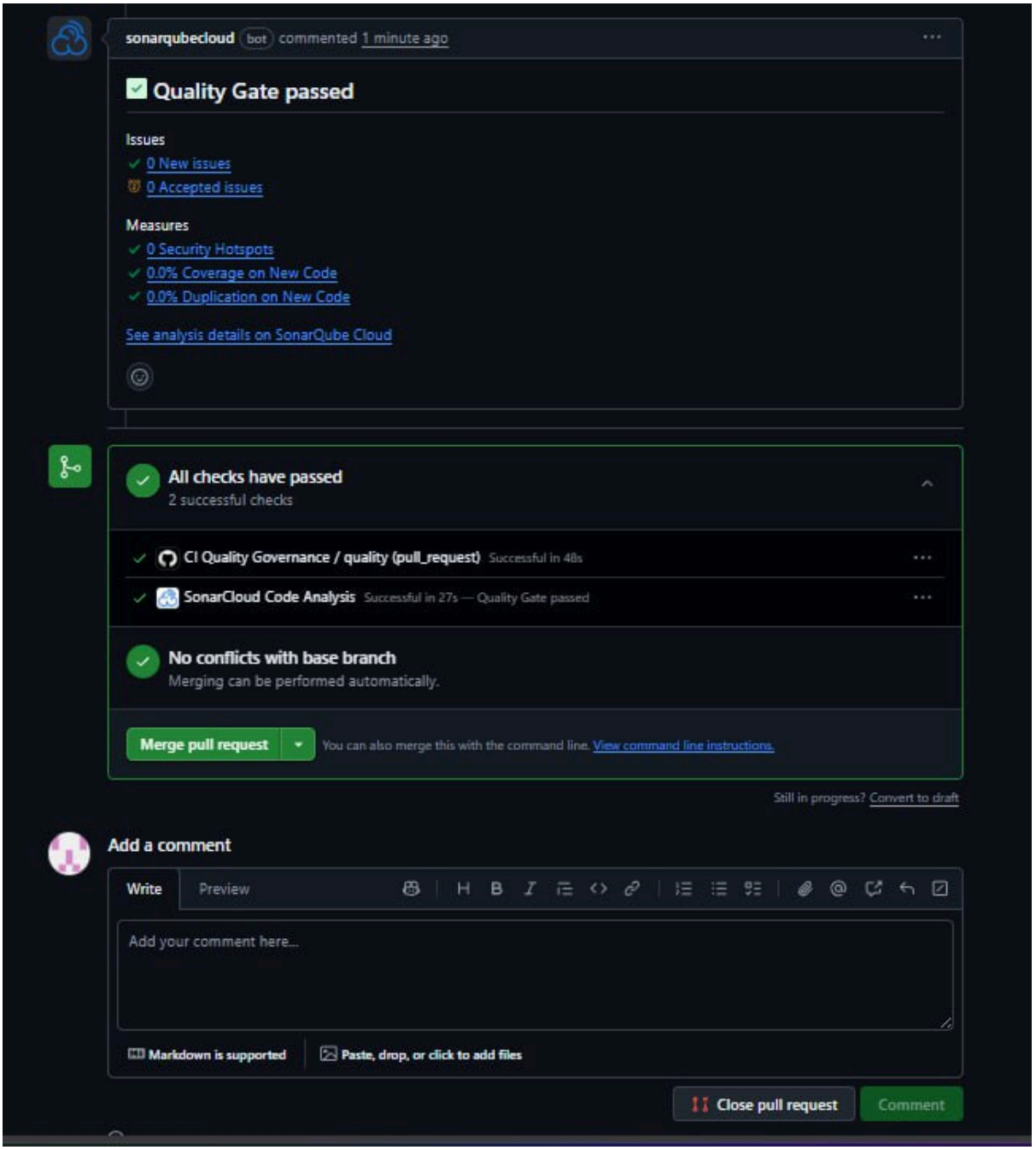
Después de la configuración de SonarQube y que ya estuviera enlazado a nuestro CI Workflow realizamos una prueba de humo para corroborar la comunicación bidireccional tanto desde Github Actions a SonarQube y en viceversa.

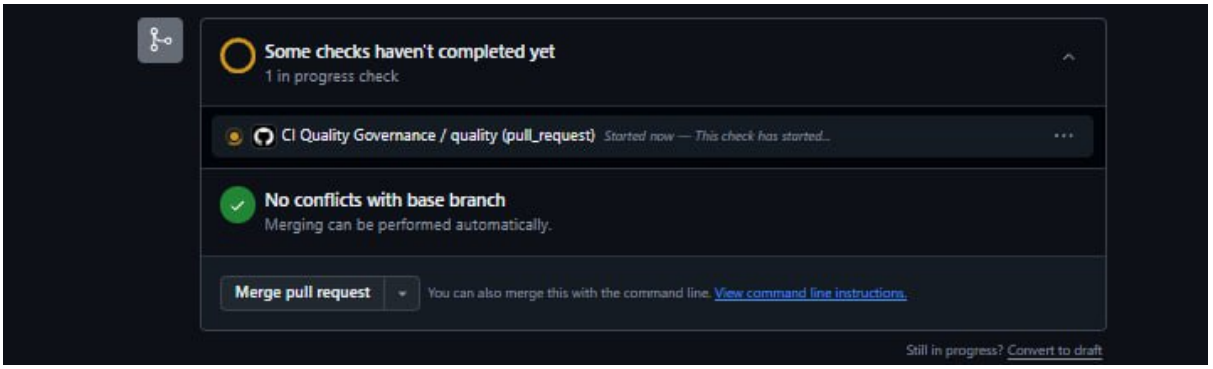
Se abrió un nuevo PR introduciendo un cambio que no afecta en nada, agregamos un archivo .txt para forzar el flujo de integración continua.

El pipeline se disparó exitosamente. SonarCloud analizó el PR y arrojó un estado de Passed (Aprobado). Dado que un archivo de texto no contiene lógica de programación, la prueba demostró que el flujo automatizado funciona correctamente: evaluó que no hubo degradación en la Cobertura de Código ni aumento en la Complejidad Ciclomática, permitiendo el merge de forma segura.



Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865





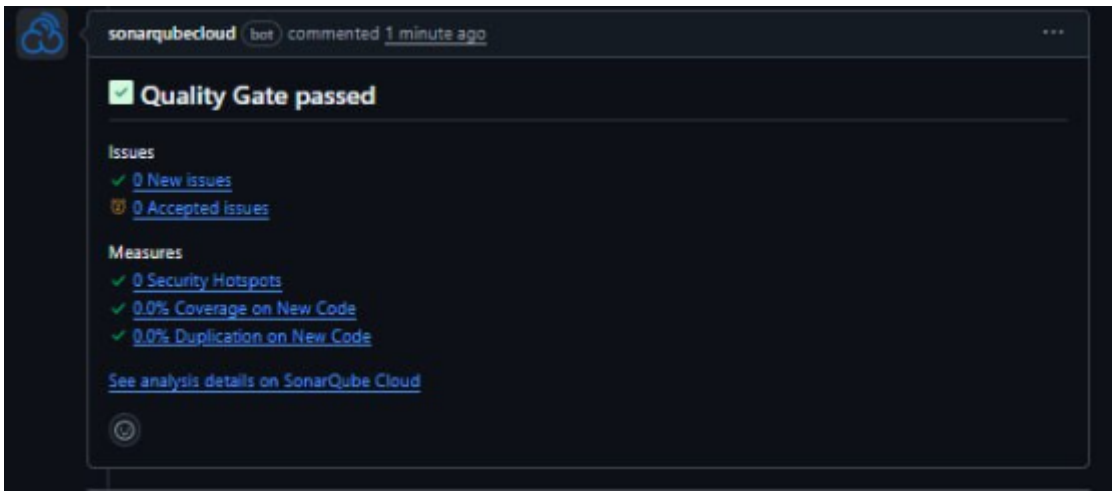
Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

Nuestros *Quality Gates* se dividen en dos métricas principales:

- **1. Quality Gate de Complejidad Ciclomática (Radon):**
  - Herramienta: Utilizamos `radon` para el análisis estático.
  - Regla: El script analiza el código fuente (`radon cc conduit -s`) y clasifica la complejidad de las funciones en letras (A, B, C, D, E, F).
  - Condición de fallo: El pipeline lee el reporte generado. Si detecta que el nuevo código introducido contiene calificaciones de alta complejidad (específicamente letras D, E o F), el flujo se interrumpe inmediatamente (retornando un *exit code 1*) y bloquea la integración del código.

- **2. Quality Gate de Cobertura de Código (Coverage):**
  - Herramienta: Utilizamos la librería `coverage` de Python.
  - Regla: Se mide el porcentaje del código que está siendo ejecutado por nuestras pruebas automatizadas.
  - Condición de fallo/advertencia: El sistema compara el resultado actual con la línea base. Si el porcentaje de cobertura de código disminuye (la calidad empeora), el *Quality Gate* emite una advertencia o hace fallar el *build*, asegurando que los nuevos cambios incluyan sus respectivas pruebas.

Iniciamos construyendo nuestros propios Quality Gates mediante scripts en GitHub Actions (apoyándonos en Radon y Coverage) para entender el flujo y bloquear código complejo. Sin embargo, para cumplir con el estándar de la industria, evolucionamos nuestro pipeline integrando SonarQube, trasladando la responsabilidad de los Quality Gates a esta plataforma especializada para un análisis mucho más profundo (seguridad, bugs y deuda técnica).



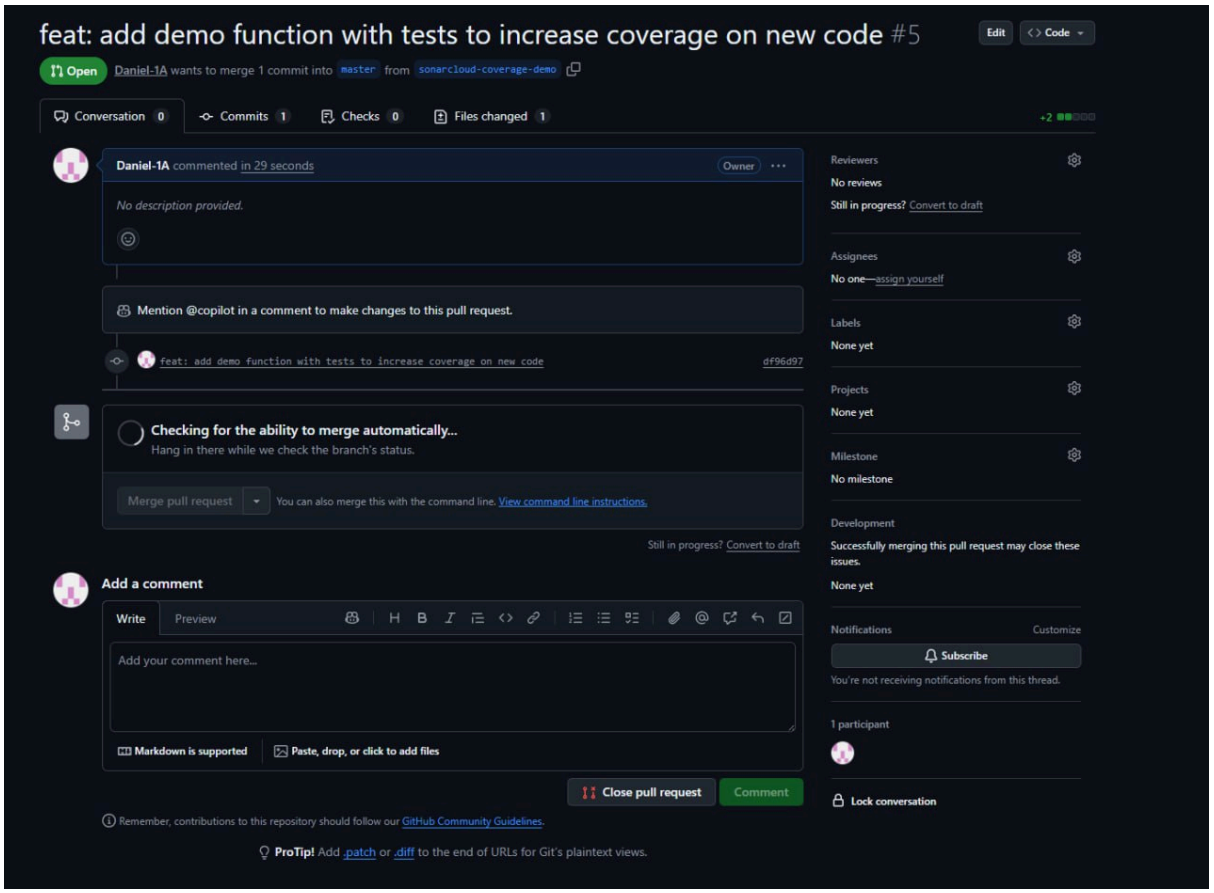
Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865

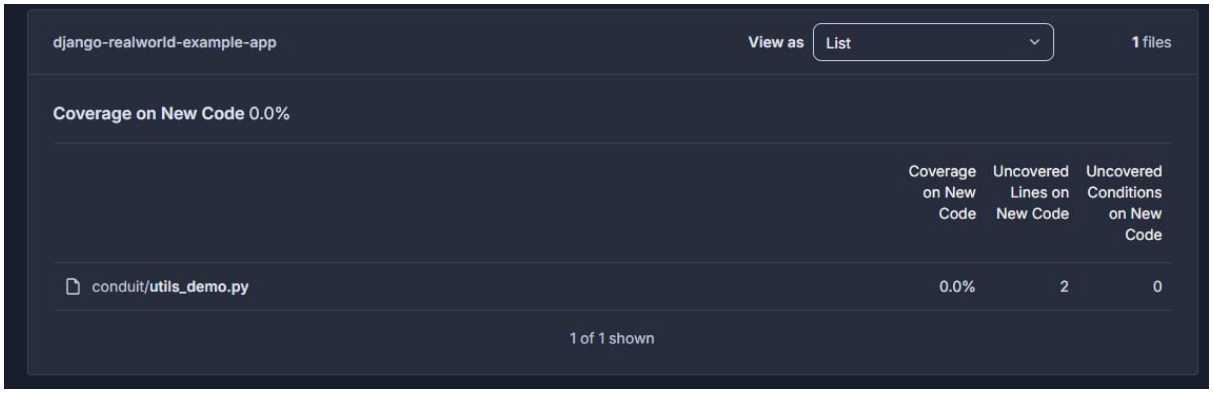
También realizamos una prueba agregando una archivo nuevo con código para ver si cambiaba algo en las métricas de los quality gates, el código que agregamos es:

conduit > utils_demo.py > is_even

```python
def is_even(n: int) -> bool:
    return n % 2 == 0
```

Y los resultados obtenidos no varían mucho con el .txt, intuimos que es por la longitud del código y como está estructurado:





Carlos Daniel Martínez García - 19001064
Douglas Daniel Pérez Hernández - 22003865